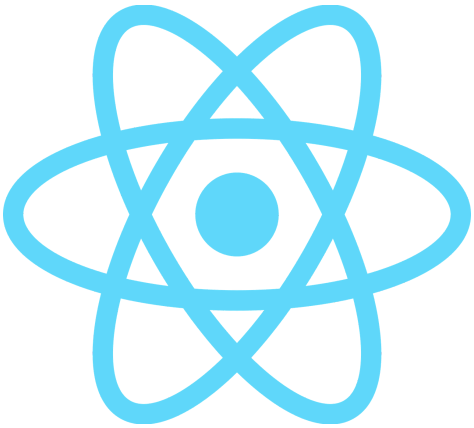
Tecnologías

Front-End

React

Es una biblioteca de JavaScript (creada por Meta) para construir interfaces de usuario basadas en componentes.

¿Por qué usarlo? Permite crear aplicaciones rápidas y escalables mediante el uso del Virtual DOM, que actualiza solo las partes de la página que cambian sin recargar todo el sitio. Es ideal para aplicaciones de una sola página (SPA) donde la interactividad es fundamental.



Back-End y Base de datos

Supabase

Se define como la alternativa de código abierto a Firebase. Es una plataforma "Backend-as-a-Service" (BaaS).

¿Por qué usarlo? Te proporciona una base de datos PostgreSQL completa, autenticación de usuarios, almacenamiento de archivos y APIs en tiempo real de forma inmediata, sin tener que configurar un servidor desde cero.

Diseño

Tailwind

Es un framework de CSS orientado a utilidades (utility-first).

¿Por qué usarlo? En lugar de escribir archivos CSS separados con nombres de clases complejos, aplicas clases directamente en tu HTML/React (como flex, pt-4, text-center). Esto acelera el diseño, asegura que el sitio sea responsivo por defecto y evita que el archivo CSS crezca descontroladamente.

Lenguaje de programación

Typescript

TypeScript es un "superconjunto" de JavaScript. Básicamente, es JavaScript con superpoderes de tipado.

En JavaScript normal, se pueden hacer cosas "prohibidas" como restarle un texto a un número y el error solo aparecerá cuando el usuario esté usando la web. En TypeScript, el

editor te detiene antes:

- JavaScript: const resultado = "hola" - 5; // Devuelve NaN en ejecución.
- TypeScript: Te marca un error en rojo inmediatamente: *"No puedes restar un número a un texto"*.

Herramienta de construcción (Build Tool)

Vite (que significa "rápido" en francés) es una herramienta de construcción (*Build Tool*) que gestiona todo tu proyecto de frontend.

Antes, cuando se guardaba un cambio en el código, herramientas como Webpack tenían que volver a leer todo el proyecto para actualizar la vista. Si el proyecto era grande, tardaba segundos preciosos.

Vite cambia las reglas:

- En Desarrollo: Solo recarga el archivo específico que modificaste. Es instantáneo.
- En Producción: Usa un empaquetador llamado *Rollup* para comprimir tu código al máximo, haciendo que tu web cargue muy rápido para el usuario final.

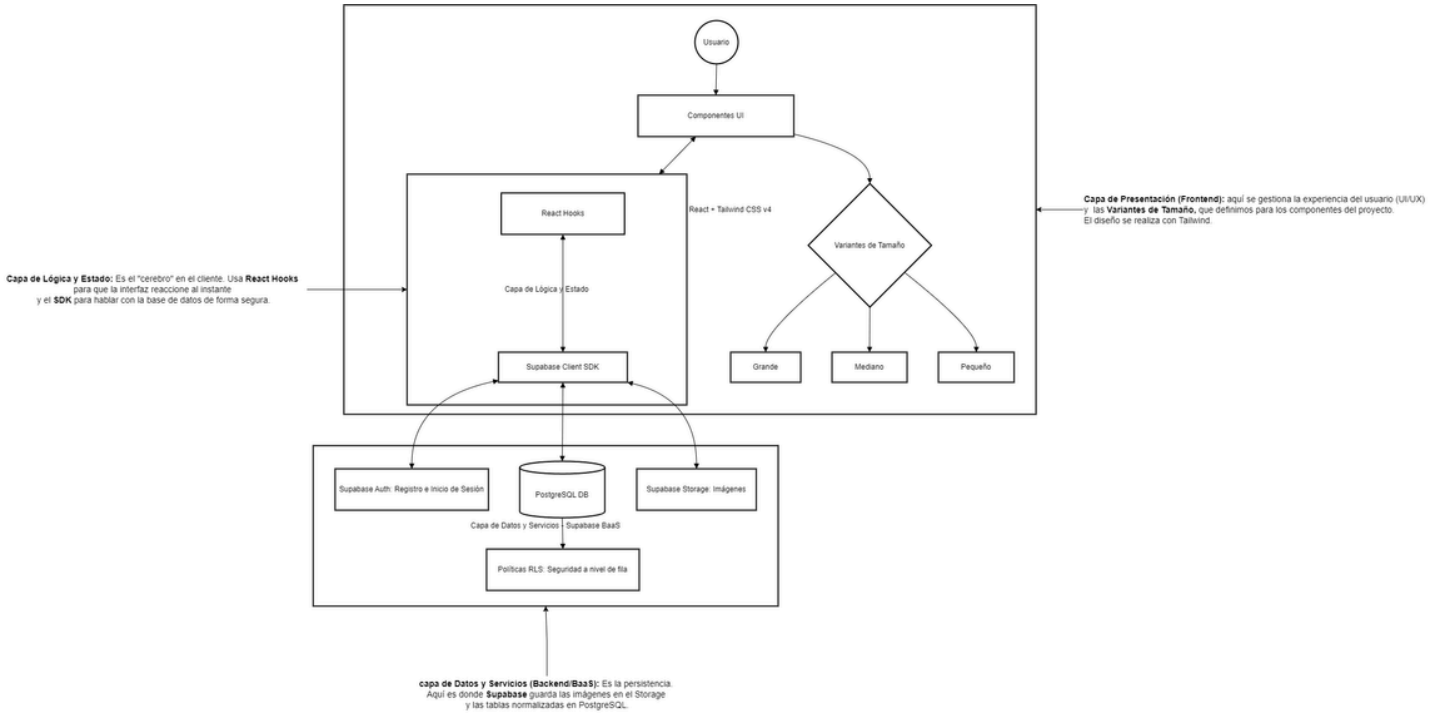
Arquitectura

Capa de Presentación (Frontend): Construida con React. Utiliza un enfoque de Componentes Basados en Propiedades para gestionar variantes definidas en Tailwind.

Capa de Lógica y Estado: Implementada mediante React Hooks (para estado local) y el Cliente de Supabase.

Capa de Datos y Servicios (BaaS): Supabase gestiona la base de datos PostgreSQL, el almacenamiento de archivos (como las fotos de 132px) y la seguridad mediante políticas de nivel de fila (RLS).

Se utiliza el **Supabase Client SDK** como capa de abstracción para la comunicación entre el cliente (React) y el servidor (BaaS), garantizando una transferencia de datos segura mediante el uso de tokens JWT y la ejecución de políticas de seguridad RLS directamente en el motor de base de datos PostgreSQL



Módulos principales

1. Módulo de Identidad y Acceso (Auth)

Este módulo gestiona la entrada y seguridad de los usuarios en la plataforma.

- Funcionalidad: Permite el auto-registro, inicio de sesión y recuperación de contraseña. Incluye la lógica de protección de rutas mediante middleware para separar los privilegios de "Usuario Estándar" y "Administrador".
- Tecnología: Utiliza Supabase Auth y políticas RLS para asegurar que cada usuario acceda solo a su información.
- Componentes: Formularios con Inputs validados y mensajes de error.

2. Módulo de Biblioteca Personal (CRUD de Ítems)

Es el corazón operativo de la aplicación, donde se gestionan los registros individuales.

- Funcionalidad: Gestión completa (crear, leer, editar, eliminar) de fichas de ítems (libros y videojuegos). El sistema asegura que el usuario solo pueda modificar los ítems de su propiedad.
- Base de Datos: Impacta directamente en la tabla Items, guardando valoraciones numéricas y descripciones.
- Componentes: formulario para crear la ficha de un nuevo ítems o editar algún ítem ya existente, botones para acciones de guardado, eliminación o edición.

3. Módulo de Administración de Catálogo

Exclusivo para el perfil de Administrador, encargado de mantener la estructura del sistema.

- Funcionalidad: Gestión centralizada de la tabla de usuarios y control total sobre los Géneros disponibles para los usuarios.
- Base de Datos: Edición directa sobre las tablas maestras Usuario, ROL y Genero.
- Componentes: Dropdowns de selección, botones para eliminar, guardar o modificar y tablas administrativas con Paginación.

4. Módulo de Gestión Multimedia y Portadas

Encargado de la parte visual y el almacenamiento de archivos del catálogo.

- Funcionalidad: Subida y asociación de imágenes (JPG/PNG) para las portadas de los ítems.

- Tecnología: Implementado mediante Supabase Storage, guardando la referencia en el campo imagen_url de la tabla Items.
- Componentes: Contenedor de Upload Photo en el formulario de creación o edición de un ítem.

5. Módulo de Navegación e Inteligencia de Búsqueda

Facilita la localización de contenidos dentro de colecciones extensas.

- Funcionalidad: Incluye una barra de búsqueda por nombre/título o autor, y un sistema de filtrado avanzado por tipo (Libro/Juego), género, fecha de inserción o puntuación.
- Lógica: Realiza consultas filtradas sobre las tablas Items y la tabla intermedia Genero_Item.
- Componentes: input de tipo text para la búsqueda y un dropdown para el filtrado.

6. Módulo de Análisis y Estadísticas (Dashboard)

Transforma los datos registrados en información visual estratégica.

- Funcionalidad: Genera métricas personalizadas para el usuario (libros/juegos por año, géneros favoritos) y métricas globales para el administrador (total de usuarios, top 5 géneros preferidos).
- Componentes: Utiliza las Tarjetas de Estadísticas en sus tres variantes (grande, mediana, pequeña) para mostrar rankings y totales de forma jerarquizada.

7. Módulo de Gestión de Perfil y Cuenta

Este módulo otorga al usuario el control total sobre su información personal y su permanencia en la plataforma, cumpliendo con los estándares de privacidad y usabilidad.

- Sub-módulo: Modificación de Perfil:
  - Funcionalidad: Permite al usuario actualizar sus datos básicos (nombre, apellidos, fecha de nacimiento).
  - Interacción con Base de Datos: Realiza operaciones de actualización (UPDATE) en la tabla Usuario filtrando por el ID de la sesión activa.
  - Componentes UI: formulario con los Inputs estándar para los campos que tendrán la información a modificar.
- Sub-módulo: Baja de Usuario (Derecho al Olvido):

- Funcionalidad: Permite al usuario eliminar (de forma lógica) su cuenta de forma definitiva.
- Lógica de Negocio: Al ejecutar la baja, el sistema realiza una desactivación del registro en la tabla Usuario.
- Seguridad: Requiere una confirmación adicional (vía modal o re-autenticación) para evitar eliminaciones accidentales.

8. Módulo de Internacionalización (i18n):

El sistema está preparado para soportar múltiples idiomas sin necesidad de duplicar código ni vistas.

- **Tecnología:** Implementado con react-i18next, extrayendo todos los textos estáticos a diccionarios de traducción (ej. t("formLogin.titulo")). Esto facilita la escalabilidad global de la bitácora.

9. Módulo de Monitorización y Telemetría (Rendimiento):

Rastreo en tiempo real de la experiencia del usuario y métricas de rendimiento en producción.

- **Tecnología:** Integración nativa de **Vercel Analytics** (para rastreo de tráfico y visitantes únicos) y **Vercel Speed Insights** para medir los Core Web Vitals (LCP, CLS) directamente desde el main.tsx.

Nuevas funcionalidades

- **Sistema Global de Notificaciones (Zustand):** Implementación de un estado global para gestionar mensajes de aviso (éxito, error, advertencias) en toda la aplicación de forma consistente, mejorando drásticamente la UX.
- **Recuperación Segura de Credenciales:** Flujo completo de restablecimiento de contraseñas vía email, incluyendo la invalidación y cierre de sesión forzoso por seguridad para evitar fugas de datos o "sesiones zombis".
- **Eliminación Lógica de Usuarios (Soft Delete):** Sistema de desactivación de cuentas que oculta al usuario del sistema pero mantiene intacta la integridad referencial de la base de datos.
- **Filtrado Dinámico en Tiempo Real:** Capacidad de la biblioteca para renderizar búsquedas compuestas (por texto y por tipo de ítem) de forma inmediata en el frontend.
- **Traducción de los ficheros:** componente botón para realizar la traducción de los ficheros.

- **Modo oscuro**: nuevo componente para el modo oscuro.
- **Optimización de Rendimiento y Core Web Vitals:** Transición de fuentes web externas a **fuentes locales (.woff2)** con la regla font-display: swap. Esto elimina los bloqueos de renderizado y reduce drásticamente los saltos visuales (CLS) y el tiempo de carga (LCP).
- **Gestión Avanzada de Rutas SPA (Single Page Application):** Configuración de un motor de reescritura de rutas en producción mediante vercel.json para evitar los errores 404 al refrescar vistas profundas (como la biblioteca o el panel de administración).
- **Protección contra "Deadlocks" en Sesiones Temporales:** Refactorización del ciclo de vida de autenticación para interceptar eventos especiales (como PASSWORD_RECOVERY). El sistema ahora es capaz de eludir consultas a la base de datos cuando el token carece de permisos RLS completos, evitando bloqueos silenciosos en la UI.
- **Accesibilidad Semántica (A11y):** Integración de etiquetas aria-label, aria-pressed y clases sr-only (Screen Reader Only) en formularios y botones de filtrado, garantizando que ListO sea navegable para usuarios que dependen de lectores de pantalla.
- **Renderizado Perezoso (Lazy Loading):** Carga diferida de componentes pesados (como los modales de diálogo o las gráficas del dashboard) para que el navegador solo descargue el código de Javascript cuando el usuario realmente lo necesita, aligerando el peso inicial (Bundle size) de la aplicación.

Planificación y normalización de la base de datos

El diseño de la base de datos se ha implementado en PostgreSQL (vía Supabase), siguiendo un modelo relacional que garantiza la integridad referencial y la escalabilidad del sistema. Se ha puesto especial énfasis en la normalización para eliminar redundancias y optimizar las consultas del frontend.

La base de datos se ha normalizado hasta la Tercera Forma Normal (3FN):

- **Primera Forma Normal (1FN):** Se garantiza que todos los atributos sean atómicos. Por ejemplo, en la tabla Usuario, los apellidos se dividen en ape1 y ape2 para facilitar búsquedas y ordenamiento.
- **Segunda Forma Normal (2FN):** Se han eliminado las dependencias parciales. Atributos como el nombre del rol o la descripción del género dependen únicamente de sus respectivas claves primarias (id_rol, id_genero) y no de las tablas principales de datos.

- **Tercera Forma Normal (3FN):** Se eliminaron las dependencias transitivas mediante el uso de tablas maestras como Tipo y Rol, asegurando que cada dato no clave dependa únicamente de la clave primaria.

Entidades y Relaciones Principales

El modelo cuenta con 6 tablas interconectadas que dan soporte a los módulos del proyecto:

- **Gestión de Usuarios y Accesos:** La tabla Usuario se relaciona de N:1 con ROL, permitiendo que los usuarios se auto-registren con un rol predeterminado, además, sirve para diferenciar entre el Administrador (control total) y el Usuario Estándar (auto-registro).
- **Catálogo de Ítems:** La tabla Items es el núcleo de la información, donde se almacenan datos como valoracion y la imagen_url (vinculada al Storage de Supabase para los elementos multimedia). Se relaciona con los usuarios, teniendo en cuenta que un usuario puede tener muchos items, pero un item es de un solo usuario.
- **Categorización Dinámica:**
  - Relación **1:N** entre Tipo e Items para clasificar el contenido de forma rígida.
  - Relación N:M (Muchos a Muchos) entre Genero e Items, resuelta mediante la tabla intermedia Genero_Item. Esto permite que un ítem (como un libro o película) pertenezca a múltiples géneros simultáneamente.

Seguridad y Consistencia

- **Integridad Referencial:** Se han definido claves foráneas (FK) con restricciones para evitar registros huérfanos.
- **Tipado de Datos:** Se utilizan tipos específicos como numeric para valoraciones e id_rol, y date para la fecha de nacimiento, optimizando el rendimiento de la base de datos.
- **Políticas RLS:** Aunque no figuran en el diagrama físico, la planificación incluye políticas de **Row Level Security** en Supabase para asegurar que los usuarios solo puedan interactuar con sus propios Items mientras el administrador mantiene acceso total.

Tablas de la Base de Datos - Modelo Entidad Relación

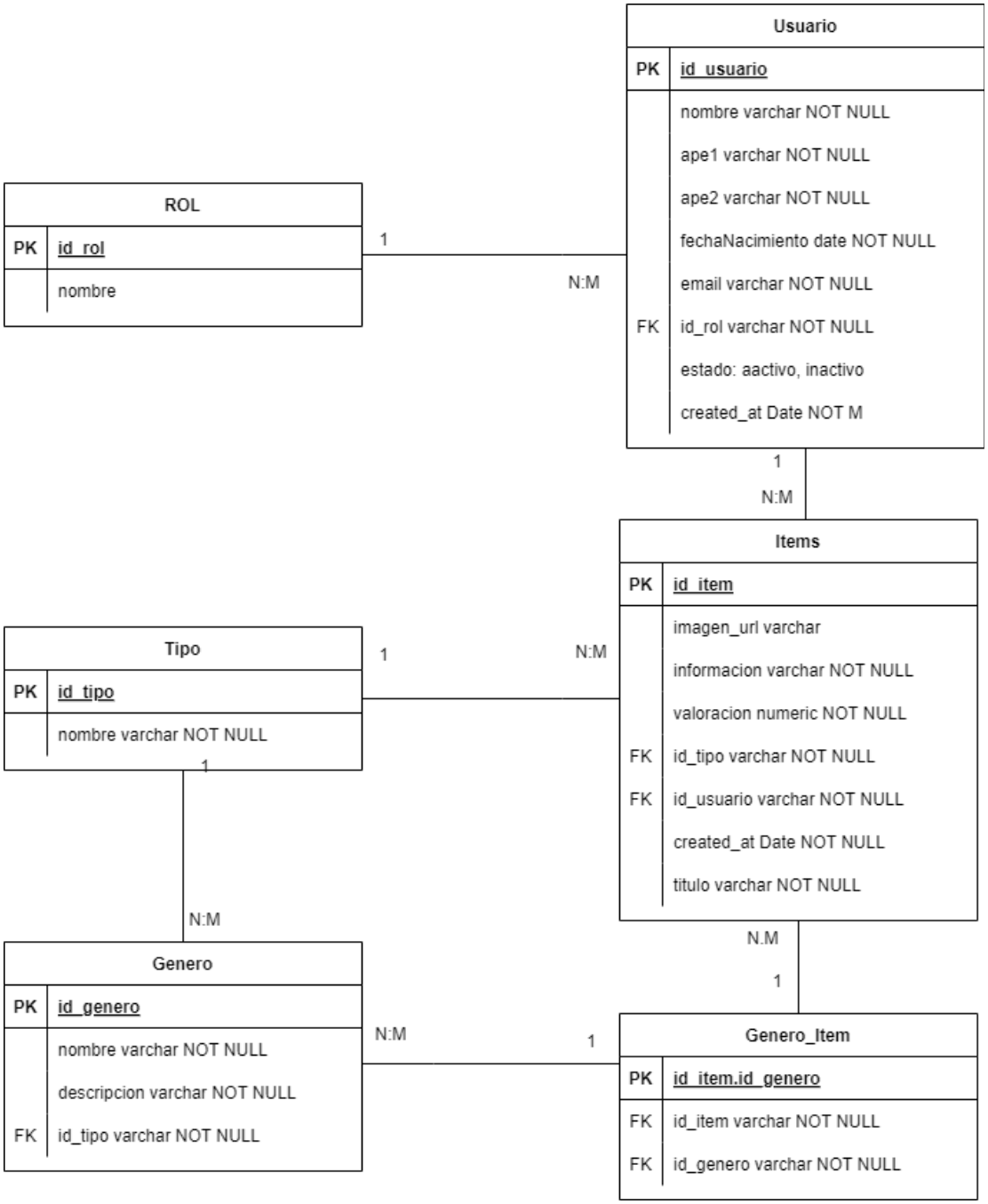


Diagrama de supabase

Para crear la base de datos hemos tenido que crear un correo y una cuenta supabase.

Retos Técnicos y Resolución de Conflictos (Troubleshooting)

Durante el ciclo de vida del desarrollo de ListO, el equipo se enfrentó a diversos desafíos técnicos que requirieron investigación, depuración y refactorización. A continuación se detallan los más relevantes:

1. Control de Versiones y Entorno de Desarrollo

- Gestión de ramas en Git: Se detectó y revirtió de forma segura una fusión (*merge*) accidental en la rama de producción (main), estableciendo flujos de trabajo más estrictos para la integración continua.
- Sincronización de Entornos Locales: Se resolvieron conflictos de dependencias e inconsistencias en el entorno de ejecución de React entre los equipos de desarrollo, lo que requirió la reestructuración del repositorio y la instalación limpia de los módulos de Node (node_modules).

2. Arquitectura Frontend y Diseño UI

- Refactorización de Componentes (React): Para mejorar la escalabilidad y el principio de responsabilidad única, se refactorizó un componente monolítico de formulario, dividiéndolo en 4 componentes Input especializados.
- Depuración de Estilos (Tailwind CSS): Se invirtió tiempo de investigación en solucionar fallos de renderizado en la interfaz, originados por errores tipográficos en las clases de utilidad (ej. corrección de sintaxis de alineación items-center).
- Sincronización de Estado y UI: Se mitigaron problemas de reactividad visual en las tablas de datos, implementando estrategias temporales de refresco automático al recuperar el foco de la ventana para mantener la coherencia de la información.

3. Gestión de Estado Global y Lógica de Negocio

- Implementación de Telemetría UI (Zustand): Se desarrolló un *Store* centralizado para gestionar el sistema global de notificaciones y retroalimentación al usuario (alertas de éxito, error, advertencia) de forma predecible.
- Optimización de Algoritmos (Dashboard): Se depuró la lógica de procesamiento de datos en el módulo de estadísticas, ajustando los algoritmos de agrupación, conteo y ordenación descendente para la correcta visualización de elementos (ej. géneros más frecuentes).

4. Backend, Base de Datos y Seguridad (Supabase)

- Aprovisionamiento de Infraestructura (BaaS): Se orquestó la creación y vinculación de las cuentas de servicio necesarias (Google y Supabase) para establecer el entorno de base de datos en la nube desde cero.
- Automatización de Datos (PostgreSQL): Se desarrollaron e implementaron *Triggers* y funciones en la base de datos para automatizar la creación de perfiles de usuario vinculados tras el registro en el sistema de autenticación.
- Políticas de Seguridad a Nivel de Fila (RLS): Se configuraron políticas restrictivas en la base de datos para garantizar la privacidad y el aislamiento de los datos entre los distintos usuarios.
- Prevención de Deadlocks y Asincronía: Se solucionaron bloqueos silenciosos ("cuelgues") en las peticiones asíncronas a Supabase al cambiar de vistas. Esto se logró mejorando la hidratación del estado y el manejo de excepciones en las conexiones para evitar el "congelamiento" de la aplicación.
- Seguridad en el Ciclo de Vida de Autenticación: Se corrigió una vulnerabilidad en el flujo de recuperación de contraseñas. La lógica fue refactorizada para forzar la invalidación de la sesión temporal (*logout*) y la redirección automática a la vista de acceso, previniendo así el mantenimiento de sesiones inestables.
- Control de Acceso Basado en Roles (RBAC): Se estabilizó el flujo de creación de perfiles desde el panel de gestión del administrador, resolviendo conflictos entre los registros manuales y los flujos de auto-registro de usuarios estándar.